

তোমার দুই কোডের সম্পূর্ণ ব্যাখ্যা — লাইন ধরে ধরে

FILE 1: Supervised (titanic.csv) · FILE 2: Unsupervised (load_wine)

প্রতিটি লাইন কী করে · কেন লাগে · PCA কখন দরকার কখন নয় · metric-গুলোর আসল মানে

⚠️ প্রথমেই দুইটা সমস্যা ধরা পড়েছে — কোড চালিয়ে পরীক্ষা করে দেখেছি

সমস্যা ১ — DROP লাইনটা কাজই করছে না। তুমি লিখেছো:

```
# DROP = ["alive", "class", "who", "adult_male"]
```

এটা শুধু comment, আর কোডের **কোথাও DROP ব্যবহার হয়নি**। মানে comment তুলে দিলেও কিছুই বাদ যাবে না, কারণ `df.drop(columns=DROP)` লাইনটাই নেই। ফল: titanic-এ **accuracy = 1.0000** আসছে, যা মিথ্যা — কারণ alive column আসলে survived -এরই yes/no রূপ, model উত্তর দেখে ফেলছে।

সমাধান (নতুন txt ফাইলে ঠিক করে দিয়েছি):

```
DROP = ["alive", "class", "who", "adult_male"] # comment তুলে দাও  
df = df.drop(columns=[c for c in DROP if c in df.columns]) # এই লাইনটা যোগ করো
```

প্রমাণ: ঠিক করার পরে accuracy 1.0000 → **0.7924**। এটাই আসল মান।

সমস্যা ২ — K = 4 বসানো, কিন্তু wine-এ class আছে ৩টা। ফল:

K	ARI	মন্তব্য
2	0.3743	কম
3	0.8951	সঠিক class সংখ্যা
4	0.6774	তোমার সেটিং

Elbow ও Silhouette দুটোই K=3 বলছে, তাই K = 3 করে দিয়েছি।

FILE 1 — SUPERVISED, লাইন ধরে ধরে

Section 1 — DATA SOURCE

```
df = pd.read_csv("titanic.csv", na_values=["?", "N/A", "NA", "na", "", " ", "missing", "-", "null", "None"])  
TARGET = "survived"
```

na_values কেন? CSV-তে ফাঁকা ঘর নানাভাবে লেখা থাকে। এটা না দিলে pandas "?" -কে সাধারণ লেখা ভাবে, তখন পুরো column টেক্রাট হয়ে যাবে — সংখ্যা আর সংখ্যা থাকবে না।

TARGET মানে column-এর নাম, ভেতরের মান নয়। "survived" column-এর নাম; ভেতরে থাকে 0 আর 1।

Section 3 — CLEAN

```
df = df.drop_duplicates().reset_index(drop=True)
```

একই row দুবার থাকলে একটা train-এ, একটা test-এ যেতে পারে। তখন model যেটা মুখস্থ করেছে সেটাতেই পরীক্ষা দিচ্ছে — এটাই **data leakage**। `reset_index(drop=True)` সারির নম্বরগুলো আবার ০,১,২... করে সাজায়; `drop=True` না দিলে পুরনো index একটা নতুন column হয়ে যোগ হয়ে যেত।

```
for c in df.columns:
    if not pd.api.types.is_numeric_dtype(df[c]):
        df[c] = LabelEncoder().fit_transform(df[c].astype(str).str.strip().str.lower())
```

sklearn শুধু সংখ্যা বোঝে, লেখা বোঝে না। "male", "female" → 1, 0 হয়ে যায়।

`.str.strip()` সামনে-পিছনের ফাঁকা জায়গা কাটে, `.str.lower()` ছোট হাতের করে — যাতে "Male", "male", " male" তিনটা আলাদা category না হয়ে একটাই হয়।

এখানে `is_numeric_dtype` ব্যবহার করেছে — এটা ঠিক আছে। কেউ কেউ লেখে `df[c].dtype == "object"`, কিন্তু নতুন pandas 3.x-এ text column-এর dtype হয় `str`, `object` নয় — তখন ওই চেক চূপচাপ ব্যর্থ হয়। তোমার লেখাটা সব version-এ কাজ করে।

```
df = df.fillna(df.median())
```

median কেন, mean কেন নয়? Median outlier-এ টলে না। ৫ জনের আয় ২০, ২২, ২৫, ২৭, ৫০০ হলে mean = ১১৮.৮ (কেউই এত আয় করে না), কিন্তু median = ২৫ — বাস্তবসম্মত। একটা চরম মানই mean-কে টেনে নেয়।

```
X, y = df.drop(columns=TARGET), df[TARGET]
TASK = "classification" if y.nunique() <= 20 else "regression"
```

X = সব feature (target বাদে), y = উত্তর।

TASK নিজে থেকে ঠিক হয়: target-এ ২০টার কম আলাদা মান থাকলে ধরে নিচ্ছি এগুলো class (classification); বেশি হলে ধরে নিচ্ছি continuous সংখ্যা (regression)। titanic-এ survived-এ মাত্র ২টা মান (0,1) → classification। tips.csv-এ tip-এ ১২৩টা মান → regression।

Section 5 — EDA

```
imp = pd.Series(RF(random_state=42).fit(X, y).feature_importances_, X.columns).sort_values()
print(imp.iloc[::-1].head(10))
```

Random Forest প্রতিটি feature কতটা কাজে লেগেছে তার score দেয় (`feature_importances_`)। `sort_values()` ছোট থেকে বড় সাজায়, তাই `.iloc[::-1]` দিয়ে উল্টে দিলে বড়টা আগে আসে। আর `imp.tail(15)` plot করা হয়েছে কারণ barh-এ নিচ থেকে উপরে আঁকে, তাই tail নিলে বড়গুলো উপরে দেখায়।

Section 6 — SPLIT + SCALE (viva-র সবচেয়ে প্রিয় জায়গা)

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42)
sc = StandardScaler()
Xtr, Xte = sc.fit_transform(Xtr), sc.transform(Xte)
```

StandardScaler সূত্র:

$$z = (x - \mu) / \sigma \quad \mu = \text{গড়}, \sigma = \text{standard deviation}$$

ফলে প্রতিটি feature-এর mean হয় 0, std হয় 1।

কেন দরকার? ধরো একটি feature fare ≈ 500 , আরেকটি sex = 0/1। SVM, KNN দূরত্ব মাপে $(x_1 - x_2)^2$ । fare-এ ১০০ পার্থক্য বর্গ করলে ১০,০০০; sex-এ ১ পার্থক্য বর্গ করলে ১। মানে **fare একাই সব সিদ্ধান্ত নেবে** — সে বেশি দরকারি বলে নয়, শুধু তার সংখ্যা বড় বলে। Scaling সবাইকে সমান সুযোগ দেয়।

fit_transform আর transform -এর পার্থক্য — এটা জিজ্ঞেস করবেই:

fit_transform(Xtr) = ① train থেকে μ, σ শেখে ② train-কে বদলায়।

transform(Xte) = শুধু আগে শেখা μ, σ দিয়ে test-কে বদলায়।

test-এ fit_transform দিলে test-এর তথ্য preprocessing-এ ঢুকে যেত = **data leakage**। Test set হলো বন্ধ খাম — predict করার আগে খোলা নিষেধ।

random_state=42 কেন? Split-টা random। এটা দিলে প্রতিবার একই split হয়, ফলে ফলাফল বারবার এক আসে আর ছয়টা model-কে হুবহু একই ডেটায় তুলনা করা যায়। ৪২ সংখ্যাটার কোনো জাদু নেই।

Section 7 — PCA

```
nc = min(N_COMP, Xtr.shape[1]) if isinstance(N_COMP, int) else N_COMP
pca = PCA(n_components=nc)
Xtr_p, Xte_p = pca.fit_transform(Xtr), pca.transform(Xte)
```

`min(N_COMP, Xtr.shape[1])` একটি নিরাপত্তা — feature-এর চেয়ে বেশি component চাওয়া যায় না। ৩টা feature থাকলে ৫টা component চাইলে error দিত; min() সেটা আটকায়।

এখানেও একই নিয়ম: PCA **train-এ fit, test-এ শুধু transform**। কারণ PCA-র দিকগুলো (component) train data থেকে শেখা।

PCA কখন দরকার, কখন দরকার নেই

PCA আসলে কী করে? ডেটা যেসব দিকে সবচেয়ে বেশি ছড়ানো (সবচেয়ে বেশি variance), সেই দিকগুলো খুঁজে বের করে এবং শুধু সেগুলো রেখে বাকিগুলো ফেলে দেয়। গাণিতিকভাবে covariance matrix-এর সবচেয়ে বড় eigenvalue-এর eigenvector-ই প্রথম principal component।

PCA দরকার যখন	PCA দরকার নেই যখন
Feature অনেক বেশি (৩০, ১০০, হাজার)	Feature মাত্র ২-৫টা
Correlation heatmap-এ গাঢ় ব্লক — feature গুলো একই জিনিস মাপছে	Feature গুলো পরস্পর স্বাধীন, কোনো পুনরাবৃত্তি নেই
2D scatter plot আঁকতে হবে	ডেটা এমনিতেই 2D
Training খুব ধীর	এমনিতেই দ্রুত চলছে
Multicollinearity সমস্যা করছে	প্রতিটি feature-এর আলাদা মানে দরকার (interpretability)

উদাহরণ ১ — PCA দরকার: Dry Bean-এ ১৬টা feature, কিন্তু Area, Perimeter, ConvexArea, EquivDiameter সবই আসলে "বিনটা কত বড়" মাপছে। PCA এগুলোকে একটি "size" দিকে চেপে দিয়ে ১৬ → ৪ করে ফেলে, অথচ accuracy প্রায় একই থাকে।

উদাহরণ ২ — PCA অকেজো: exam-এর make_blobs ডেটা এমনিতেই ২ dimension-এর। সেখানে PCA(2) দিলে variance ১০০% থাকে, before/after ফলাফল হুবহু এক (+0.0000)। শিক্ষক জিজ্ঞেস করলে বলবে: "ডেটা আগে থেকেই 2D, তাই PCA-র চেপে দেওয়ার মতো কিছু ছিল না।"

PCA-র দাম কী? ① Component গুলোর আর কোনো নাম-অর্থ থাকে না — PC1 মানে কী, বলা কঠিন (interpretability হারায়)। ② কিছু তথ্য হারায়, তাই accuracy সামান্য কমতে পারে। ③ **Scaling ছাড়া PCA করলে ফল ভুল হবে**, কারণ PCA variance দেখে আর বড় unit-এর feature-এর variance এমনভাবেই বড়।

Metric গুলোর আসল মানে

Confusion Matrix

		Predicted	
		0	1
Actual	0	130	15
	1	34	57

সারি = আসল, কলাম = model যা বলেছে। কোণাকুণি ঘরগুলো ঠিক, বাকি ভুল।

উপরে: ১৩০ জন সত্যিই মরেছে এবং model-ও তাই বলেছে (TN)। ৫৭ জন বেঁচেছে ও model তাই বলেছে (TP)। ১৫ জনকে ভুল করে "বেঁচেছে" বলেছে (FP)। ৩৪ জনকে ভুল করে "মরেছে" বলেছে (FN)।

Metric	সূত্র	বাংলায় প্রশ্নটা	কখন দরকার
Accuracy	$(TP+TN)/\text{মোট}$	মোট কতটা ঠিক বললাম?	class সমান হলে
Precision	$TP/(TP+FP)$	যাদের "বাঁচবে" বলেছি, তাদের কতজন সত্যিই বেঁচেছে?	ভুল alarm-এর খরচ বেশি (spam filter)
Recall	$TP/(TP+FN)$	যারা সত্যিই বেঁচেছে, তাদের কতজনকে ধরতে পারলাম?	মিস করার খরচ বেশি (রোগ নির্ণয়)
F1	$2PR/(P+R)$	দুটোর ভারসাম্য	class অসমান হলে

Accuracy একা কেন যথেষ্ট নয়? ১০০ জনের ৯৯ জন সুস্থ, ১ জন অসুস্থ। Model সবাইকে "সুস্থ" বললে accuracy = ৯৯% — অথচ একটা রোগীও ধরেনি! তাই তোমার কোডে `average="weighted"` দিয়ে F1, Precision, Recall-ও বের করা হয়েছে। **weighted** মানে প্রতিটি class-এর metric আলাদা হিসাব করে class-এর আকার অনুযায়ী গড় করা।

Regression-এর metric

Metric	মানে	ভালো কোনটা
R²	target-এর কত ভাগ ওঠানামা model ব্যাখ্যা করতে পারলো	বেশি (সর্বোচ্চ ১)
MSE	ভুলের বর্গের গড়	কম
RMSE	$\sqrt{\text{MSE}}$ — target-এর একই unit-এ ফিরে আসে, তাই বোঝা সহজ	কম
MAE	ভুলের পরম মানের গড়	কম
MAPE	শতকরায় ভুল	কম

RMSE না MAE? RMSE ভুলকে বর্গ করে, তাই **বড় ভুলকে বেশি শাস্তি** দেয়। বড় ভুল বেশি ক্ষতিকর হলে RMSE দেখো; সব ভুল সমান গুরুত্বের হলে MAE।

তোমার কোডে plot-এ সব metric একসাথে আসে। মনে রেখো — **R²-তে বেশি ভালো, কিন্তু MSE/RMSE/MAE/MAPE-তে কম ভালো।** RMSE-র bar বেড়ে গেলে সেটা উন্নতি নয়, অবনতি।

FILE 2 — UNSUPERVISED, লাইন ধরে ধরে

junk() function

```
def junk(c):
    num = pd.api.types.is_numeric_dtype(df[c])
    return (df[c].nunique() <= 1 or not num and (df[c].nunique() == len(df) or
        pd.to_datetime(df[c], errors="coerce", format="mixed").notna().mean() > .8))
```

তিন ধরনের একেজো column ধরে:

- `nunique() <= 1` → **constant column**। সবার একই মান, তাই আলাদা করার ক্ষমতা শূন্য। উপরন্তু $\sigma = 0$ হওয়ায় StandardScaler-এ ভাগ করতে সমস্যা।
- `nunique() == len(df)` এবং text → **ID column**। প্রতিটি মান আলাদা, শুধু নাম, কোনো তথ্য নেই।
- `to_datetime(...).notna().mean() > .8` → **date column**। ৮০%-এর বেশি মান তারিখ হিসেবে পড়া গেলে ধরে নিচ্ছি এটা তারিখ।

`errors="coerce"` মানে যেটা তারিখ হিসেবে পড়া যাবে না সেটা NaT (ফাঁকা) করে দাও, error দিও না।

খেয়াল করো, ID চেকে not num শর্ত আছে — অর্থাৎ শুধু text column-এর ক্ষেত্রেই "সব মান আলাদা" দেখে ID ধরা হয়। এটা খুব গুরুত্বপূর্ণ, কারণ continuous সংখ্যার column-এও প্রায়ই সব মান আলাদা থাকে। শর্তটা না থাকলে আসল feature-ও বাদ পড়ে যেত।

label সরানো

```
y = None
if LABEL is not None and LABEL in df.columns:
    y = df[LABEL]; df = df.drop(columns=LABEL)
```

এটাই supervised-এর সাথে মূল পার্থক্য। Clustering algorithm label দেখতে পাবে না — সে নিজে থেকেই দল বানাবে। label-টা শুধু `y` -তে সরিয়ে রাখা হচ্ছে যাতে শেষে **ARI দিয়ে মিলিয়ে দেখা যায়** সে কতটা ঠিক করলো।

Viva-তে জিজ্ঞেস করতে পারে: "label যখন আছেই, একে unsupervised বলছো কেন?" — উত্তর: **algorithm training-এ label দেখেনি; আমরা শুধু ফলাফল যাচাই করতে ব্যবহার করেছি।** label না থাকলে `LABEL = None` দিলেই চলে, তখন ARI-র বদলে Silhouette দিয়ে মাপা হয়।

Elbow Method

```
x1, y1, x2, y2 = ks[0], wcss[0], ks[-1], wcss[-1]
line = ((y2 - y1) ** 2 + (x2 - x1) ** 2) ** .5
dist = [abs((y2-y1)*k - (x2-x1)*w + x2*y1 - y2*x1) / line for k, w in zip(ks, wcss)]
best_k = ks[dist.index(max(dist))]
```

WCSS (Within-Cluster Sum of Squares, কোডে `inertia_`) = প্রতিটি point তার centroid থেকে কত দূরে, তার বর্গের যোগফল।

সমস্যা: K বাড়ালে WCSS **সবসময়** কমে — K = n হলে প্রতিটি point নিজেই একটি cluster, WCSS = 0। তাই সবচেয়ে কম WCSS দেখে K বাছা অর্থহীন।

সমাধান: খুঁজি সেই K যেখানে কমার হার **হঠাৎ ধীর** হয়ে যায় — গ্রাফের কনুই। গাণিতিকভাবে: প্রথম বিন্দু (K=1) থেকে শেষ বিন্দু (K=10) পর্যন্ত সরলরেখা টেনে দেখি কোন বিন্দু **ওই রেখা থেকে সবচেয়ে দূরে**। উপরের সূত্রটা স্থানাঙ্ক জ্যামিতির চেনা "**বিন্দু থেকে সরলরেখার লম্ব দূরত্ব**" সূত্র।

Clustering metric

Metric	মানে	পরিসর
Silhouette	point নিজের cluster-এর কাছে না অন্য cluster-এর কাছে	-1 থেকে +1, বেশি ভালো
ARI	আসল label-এর সাথে কতটা মিললো	0 = random, 1 = perfect
Clusters	কয়টা দল পেলো	—
Noise	DBSCAN কয়টা point-কে outlier বললো (-1)	—

ARI কেন, accuracy কেন নয়? Clustering-এর দেওয়া নম্বরগুলো অর্থহীন — cluster "0" মানে class 0 নয়; সে হয়তো class 2-কে 0 নাম দিয়েছে। Accuracy সরাসরি নাম মেলায় বলে ভুল ফল দেবে। **ARI নাম দেখে না, দেখে কারা কারা একসাথে আছে** — তাই এটাই সঠিক।

Silhouette কেন 0 করে দেওয়া হয় যখন cluster ১টা? Silhouette-এর হিসাবে "নিজের cluster" আর "সবচেয়ে কাছের অন্য cluster" — দুটোই লাগে। একটাই cluster থাকলে "অন্য cluster" নেই, তাই হিসাব করা অসম্ভব; কোড error এড়াতে 0 বসিয়ে দেয়।

চারটা algorithm

Algorithm	কীভাবে কাজ করে	কখন জেতে
KMeans	K টা centroid বসিয়ে বারবার সরায়, সবাই নিকটতম centroid-এ যায়	গোল, আলাদা cluster
Hierarchical	প্রতিটি point আলাদা cluster ধরে শুরু, ধাপে ধাপে জোড়া লাগায় → dendrogram	K জানা না থাকলে
GMM	প্রতিটি cluster একটি Gaussian; probability দিয়ে assign (soft)	Overlapping , ডিম্বাকৃতি cluster
DBSCAN	eps ব্যাসার্ধে min_samples থাকলে core point; ঘনত্ব ধরে ছড়ায়	অদ্ভুত আকৃতি, outlier আছে

n_init=10 কেন? KMeans random জায়গা থেকে শুরু করে, তাই local minimum-এ আটকাতে পারে। ১০ বার ভিন্ন শুরু থেকে চালিয়ে সবচেয়ে কম WCSS-ওয়ালা ফল রাখে।

linkage="ward" কী? Hierarchical-এ দুই cluster জোড়া লাগানোর নিয়ম। Ward সেই জোড়াটা বেছে নেয় যেটাতে within-cluster variance সবচেয়ে কম বাড়ে — অর্থাৎ K-Means-এর দর্শনের সবচেয়ে কাছের linkage।

DBSCAN-এ eps আর min_samples : eps = কত দূর পর্যন্ত প্রতিবেশী খুঁজবে, min_samples = ওই ব্যাসার্ধে কমপক্ষে কতজন থাকলে "ঘন" বলবে। eps খুব ছোট দিলে সব noise, খুব বড় দিলে সব এক cluster। তাই কোডে মন্তব্য আছে ০.২ / ০.৫ চেষ্টা করতে।

Dendrogram কেন ৩০০ sample?

```
n_dendro = min(300, len(X_after))
picked = np.random.default_rng(42).choice(len(X_after), n_dendro, replace=False)
```

১৩,০০০ leaf-এর dendrogram কালো দাগ ছাড়া কিছুই দেখায় না। তাই ৩০০টা random sample নিয়ে আঁকা হয় — **clustering কিন্তু পুরো ডেটাতেই হয়**, শুধু ছবিটা sample দিয়ে। `replace=False` মানে একই point দুবার নেওয়া হবে না।

শেষ কথা — viva-তে উত্তর দেওয়ার কার্তামো

শিক্ষক যেকোনো section দেখিয়ে "এটা কী করছে?" জিজ্ঞেস করলে এই চার ধাপে বলো:

1. **কী করছে** — "feature গুলো scale করছে"

2. **কীভাবে** — " $(x-\mu)/\sigma$ সূত্রে, mean 0 std 1 করে"

3. **কেন** — "fare ৫০০ আর sex 0/1; scale না করলে দূরত্বের হিসাবে fare একাই সব ঠিক করতো"

4. **প্রমাণ** — "output-এ shape দেখুন, scaled row গুলো দেখুন"

আর জানা না থাকলে বানিয়ে বোলো না — "নিশ্চিত নই, তবে মনে হয় এটা ... কারণ ..." বলাই ভালো।